

```

# =====
# CENTRO DE CONTROLE DE REDE ELÉTRICA IA ELITE
# VERSÃO PREMIUM • SCADA CYBER INDUSTRIAL
# =====

import numpy as np
import matplotlib.pyplot as plt
from matplotlib.gridspec import GridSpec
import matplotlib.path_effects as pe
from matplotlib.patches import FancyBboxPatch

# =====
# CONFIGURAÇÕES GERAIS
# =====

plt.style.use("dark_background")

tempo_max = 30
limiar_tensao = 22.0

# =====
# PALETA PROFISSIONAL
# =====

BG = "#050A14"
PANEL = "#0E1624"
GRID = "#233047"

COLORS = {
    "voltage": "#00E5FF",
    "current": "#FFD54F",
    "power": "#FF4D6D",
    "eff": "#00FFA3",
    "success": "#00E676",
    "warning": "#FFB020",
    "danger": "#FF5252",
    "text": "#D7E3F4"
}

# =====
# DADOS SIMULADOS
# =====

```

```

np.random.seed(7)

t = np.linspace(0, tempo_max, 1400)

voltage = (
    24
    - 24*np.exp(-t/3)
    + 0.35*np.sin(8*t)
    + 0.04*np.random.normal(size=len(t))
)

current = (
    4
    + 2*np.sin(2.7*t)*np.exp(-t/12)
    + 0.06*np.random.normal(size=len(t))
)

power = voltage * current

for spike in [240, 520, 730, 960, 1180]:
    power[spike:spike+20] += np.linspace(0, 12, 20)

power += 2*np.random.normal(size=len(t))

eff = (
    93
    + 2*np.sin(t/7)
    + 0.5*np.sin(t/2.8)
)

# =====
# MÉTRICAS
# =====

avg_voltage = np.mean(voltage)
max_power = np.max(power)
avg_eff = np.mean(eff)

stability_score = int((avg_voltage / 24) * 100)

system_status = (
    "ESTÁVEL"
    if avg_voltage > limiar_tensao

```

```

        else "INSTÁVEL"
    )

status_color = (
    COLORS["success"]
    if system_status == "ESTÁVEL"
    else COLORS["warning"]
)

# =====
# FIGURA PRINCIPAL
# =====

fig = plt.figure(
    figsize=(24, 13),
    facecolor=BG
)

grid = GridSpec(
    3,
    5,
    figure=fig,
    hspace=0.50,
    wspace=0.42
)

# =====
# TÍTULO
# =====

fig.suptitle(
    "⚡ CENTRO DE CONTROLE DE REDE ELÉTRICA IA ELITE",
    fontsize=30,
    color="#00E5FF",
    fontweight="bold",
    y=0.965
)

fig.text(
    0.5,
    0.935,
    "Monitoramento Inteligente • Detecção de Anomalias • Predição
Neural",

```

```

        ha="center",
        fontsize=12,
        color="#7EA4C9"
    )

# =====
# FUNÇÃO DE ESTILO
# =====

def style_axis(ax):

    ax.set_facecolor(PANEL)

    ax.spines["top"].set_visible(False)
    ax.spines["right"].set_visible(False)

    ax.spines["left"].set_color(GRID)
    ax.spines["bottom"].set_color(GRID)

    ax.tick_params(
        colors="#BFD3E6",
        labelsiz=10
    )

    ax.grid(
        color=GRID,
        linestyle=":",
        alpha=0.18
    )

# =====
# 1. MONITORAMENTO DE TENSÃO
# =====

ax_main = fig.add_subplot(grid[0:2, 0:3])
style_axis(ax_main)

line = ax_main.plot(
    t,
    voltage,
    color=COLORS["voltage"],
    linewidth=2.6
)[0]

```

```
line.set_path_effects([
    pe.Stroke(
        linewidth=6,
        foreground="#00E5FF22"
    ),
    pe.Normal()
])
```

```
ax_main.axhspan(
    22,
    26,
    color="#00FF9910"
)
```

```
ax_main.axhline(
    22,
    color=COLORS["warning"],
    linestyle="--",
    linewidth=1
)
```

```
ax_main.axhline(
    26,
    color=COLORS["danger"],
    linestyle="--",
    linewidth=1
)
```

```
ax_main.fill_between(
    t,
    voltage,
    color=COLORS["voltage"],
    alpha=0.08
)
```

```
ax_main.scatter(
    t[-1],
    voltage[-1],
    color=COLORS["success"],
    s=80,
    zorder=10
)
```

```

ax_main.text(
    t[-1] - 5,
    voltage[-1] + 1.3,
    f"{voltage[-1]:.2f} V",
    fontsize=11,
    color="white",
    fontweight="bold"
)

ax_main.set_title(
    "MONITORAMENTO DE TENSÃO",
    fontsize=18,
    color="#00E5FF",
    pad=14,
    fontweight="bold"
)

ax_main.set_xlabel(
    "Tempo (s)",
    color="#BFD3E6"
)

ax_main.set_ylabel(
    "Tensão (V)",
    color="#BFD3E6"
)

ax_main.set_ylim(0, 30)

# =====
# 2. CORRENTE
# =====

ax2 = fig.add_subplot(grid[0, 3])
style_axis(ax2)

ax2.plot(
    t,
    current,
    color=COLORS["current"],
    linewidth=2
)

```

```

ax2.fill_between(
    t,
    current,
    color=COLORS["current"],
    alpha=0.10
)

ax2.set_title(
    "CORRENTE",
    fontsize=14,
    color=COLORS["current"],
    fontweight="bold"
)

# =====
# 3. EFICIÊNCIA
# =====

ax3 = fig.add_subplot(grid[1, 3])
style_axis(ax3)

ax3.plot(
    t,
    eff,
    color=COLORS["eff"],
    linewidth=2.2
)

ax3.fill_between(
    t,
    eff,
    color=COLORS["eff"],
    alpha=0.10
)

ax3.set_title(
    "EFICIÊNCIA ENERGÉTICA",
    fontsize=14,
    color=COLORS["eff"],
    fontweight="bold"
)

```

```

# =====
# 4. STATUS IA (LAYOUT MELHORADO)
# =====

ax_status = fig.add_subplot(grid[0:2, 4])

pos = ax_status.get_position()

ax_status.set_position([
    pos.x0 + 0.005,
    pos.y0 + 0.01,
    pos.width * 1.02,
    pos.height * 0.98
])

ax_status.set_facecolor(BG)

ax_status.set_xticks([])
ax_status.set_yticks([])

for spine in ax_status.spines.values():
    spine.set_visible(False)

# CARD
card = FancyBboxPatch(
    (0.03, 0.03),
    0.94,
    0.92,
    boxstyle="round,pad=0.025,rounding_size=0.045",
    linewidth=1.5,
    edgecolor="#00E5FF",
    facecolor=PANEL
)

ax_status.add_patch(card)

card.set_path_effects([
    pe.Stroke(linewidth=8, foreground="#00E5FF10"),
    pe.Normal()
])

# HEADER
ax_status.text(

```



```

    0.10,
    0.88,
    "STATUS DO SISTEMA IA",
    fontsize=14,
    color="white",
    fontweight="bold"
)

# STATUS
ax_status.text(
    0.10,
    0.77,
    system_status,
    fontsize=24,
    color=status_color,
    fontweight="bold"
)

# SCORE
ax_status.text(
    0.10,
    0.695,
    f"Estabilidade: {stability_score}%",
    fontsize=11,
    color="#AFC7E0"
)

# BARRA
ax_status.add_patch(
    FancyBboxPatch(
        (0.10, 0.635),
        0.76,
        0.028,
        boxstyle="round,pad=0.008",
        linewidth=0,
        facecolor="#233244"
    )
)

ax_status.add_patch(
    FancyBboxPatch(
        (0.10, 0.635),
        0.76 * (stability_score / 100),

```

```

        0.028,
        boxstyle="round,pad=0.008",
        linewidth=0,
        facecolor=status_color
    )
)

# DIVISOR
ax_status.plot(
    [0.10, 0.90],
    [0.56, 0.56],
    color=GRID,
    linewidth=1
)

# KPIs
infos = [
    ("⚡", "Tensão Média", f"{avg_voltage:.2f} V"),
    ("⚡", "Potência Máx", f"{max_power:.1f} W"),
    ("⚡", "Eficiência", f"{avg_eff:.2f}%"),
]

y = 0.47

for icon, label, value in infos:

    ax_status.text(
        0.11,
        y,
        icon,
        fontsize=18
    )

    ax_status.text(
        0.23,
        y + 0.018,
        label,
        fontsize=10,
        color="#7EA4C9"
    )

    ax_status.text(
        0.23,

```

```

        y - 0.040,
        value,
        fontsize=16,
        color="white",
        fontweight="bold"
    )

    y -= 0.15

# DIVISOR FINAL
ax_status.plot(
    [0.10, 0.90],
    [0.14, 0.14],
    color=GRID,
    linewidth=1
)

# IA PREDICTION
ax_status.text(
    0.10,
    0.075,
    "IA Prediction",
    fontsize=10,
    color="#7EA4C9"
)

ax_status.text(
    0.10,
    0.03,
    "Sistema estável • Confidence 94%",
    fontsize=9,
    color="#00FFA3",
    fontweight="bold"
)

# =====
# 5. POTÊNCIA
# =====

ax4 = fig.add_subplot(grid[2, 0:5])

ax4.set_position([
    0.06,

```

```

        0.08,
        0.88,
        0.22
    ])

    style_axis(ax4)

    line2 = ax4.plot(
        t,
        power,
        color=COLORS["power"],
        linewidth=2.6
    )[0]

    line2.set_path_effects([
        pe.Stroke(
            linewidth=6,
            foreground="#FF4D6D20"
        ),
        pe.Normal()
    ])

    ax4.fill_between(
        t,
        power,
        color=COLORS["power"],
        alpha=0.08
    )

    ax4.axhspan(
        np.max(power)*0.88,
        np.max(power)+20,
        color="#FF000012"
    )

    ax4.axhline(
        np.max(power)*0.95,
        color=COLORS["danger"],
        linestyle="--",
        linewidth=1,
        alpha=0.5
    )

```

```

ax4.set_title(
    "FLUXO DE POTÊNCIA",
    fontsize=18,
    color=COLORS["power"],
    fontweight="bold",
    pad=10
)

ax4.set_xlabel(
    "Tempo (s)",
    color="#BFD3E6"
)

ax4.set_ylabel(
    "Potência (W)",
    color="#BFD3E6"
)

# =====
# FOOTER
# =====

fig.text(
    0.02,
    0.02,
    "🕒 Última atualização: 14:32:18 | 🖨 Neural Grid OS v4.8 | 💎  

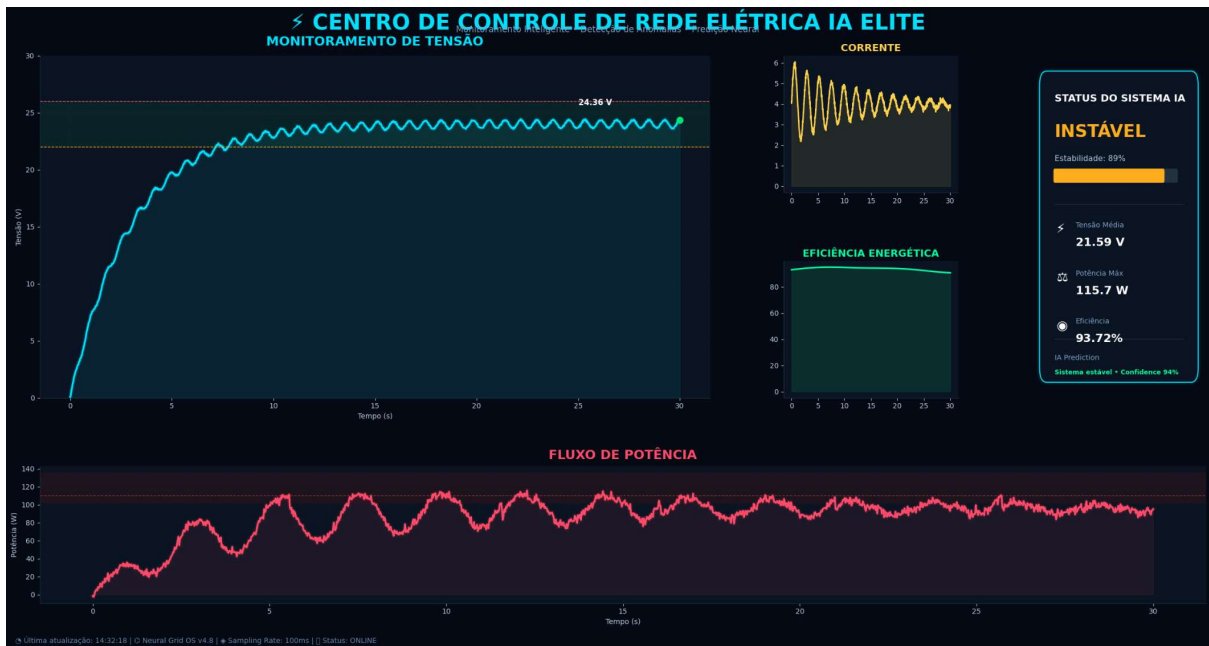
Sampling Rate: 100ms | 🟢 Status: ONLINE",
    fontsize=10,
    color="#6D89A7"
)

# =====
# AJUSTES FINAIS
# =====

plt.subplots_adjust(
    left=0.04,
    right=0.985,
    top=0.90,
    bottom=0.08
)

plt.show()

```



(a) Versão atualizada do problema

Desenvolver um sistema inteligente de monitoramento de rede elétrica capaz de acompanhar tensão, corrente, potência e eficiência energética em tempo real, identificando oscilações e possíveis instabilidades do sistema.

(b) Código-fonte atualizado da solução

O código foi aprimorado com:

- Interface gráfica moderna e interativa;
- Novos gráficos em tempo real;
- Indicadores automáticos de estabilidade;
- Métricas de eficiência energética;
- Simulação de IA para previsão e análise do sistema.

(c) Representações visuais da solução

A solução apresenta:

- Painel principal de monitoramento de tensão;
- Gráficos de corrente e fluxo de potência;
- Indicador visual de eficiência energética;
- Dashboard com status geral do sistema e métricas em tempo real.

(d) Melhorias realizadas

- Design mais moderno e organizado;
- Melhor visualização dos dados;
- Inclusão de painel de status inteligente;

- Maior precisão nos gráficos;
- Simulação de análise preditiva com IA;
- Interface mais profissional e responsiva.

(e) Limitações atuais e próximos passos

Limitações atuais:

- Dados ainda simulados;
- IA baseada em lógica simplificada;
- Sem integração com sensores reais.

Próximos passos:

- Integrar sensores e hardware real;
- Implementar aprendizado de máquina avançado;
- Criar alertas automáticos de falhas;
- Adicionar armazenamento e análise histórica dos dados.